





How long have we been addicted to alcohol?

One of the first organisms that humans domesticated was yeast. In 2011, while excavating an old graveyard in an Armenian cave, scientists discovered a 6,000 year-old winery, complete with a wine press, fermentation vessels, and even drinking cups. This winery was a major technological innovation that required understanding how to control ***Saccharomyces***, the genus of yeast used in alcohol and bread production.

Yet our interest in alcohol may date back much farther than 6,000 years. In 2008, scientists discovered that pen-tailed tree shrews (see figure below), which are similar to the ancient ancestors of all primates, are alcoholics. Their beverage of choice? A “palm wine” produced by the flowers of Bertram palms and naturally fermented by *Saccharomyces* yeast living on the flowers. This finding suggests that our own taste for alcohol may have a genetic basis that predates the Armenian wine cave by millions of years!



Figure: The pen-tailed tree shrew.



Pound for pound, the amount of palm wine that tree shrews consume would be lethal to most mammals. Fortunately, the shrews have developed efficient ways of metabolizing alcohol, and so they avoid inebriation, which would increase their risk of being killed by a predator. Because of tree shrews' alcohol tolerance, scientists believe that alcohol probably offers the shrews some evolutionary advantages, such as protection against heart attack. It is also possible that our more recent primate ancestors were also heavy drinkers — after all, chimpanzees drink naturally brewed fruit nectar — and that we may have inherited an ancestral association of alcohol intake with caloric gain.





The diauxic shift

The species of yeast that we will consider in this chapter is *Saccharomyces cerevisiae*, which can brew wine because it converts the **glucose** found in fruit into **ethanol**. We will therefore begin with a simple question: if *S. cerevisiae* often lives on grapevines, why must crushed grapes be stored in tightly sealed barrels in order to make wine?

Once its supply of glucose runs out, *S. cerevisiae* must do something to survive. It therefore inverts its metabolism, with the ethanol that it just produced becoming its new food supply. This metabolic inversion, called the **diauxic shift**, can only occur in the presence of oxygen. Without oxygen, *S. cerevisiae* hibernates until either glucose or oxygen becomes available. In other words, if winemakers don't seal their barrels, then the yeast in the barrel will metabolize the ethanol that it just produced, ruining the wine.

The diauxic shift is a complex process that affects the expression of many genes. Accordingly, it must derive from a major evolutionary event that equipped the *Saccharomyces* ancestor with a formidable advantage over its competitors: not only can *Saccharomyces* kill its competitors by producing ethanol, which is toxic to most bacteria and other yeasts, but it can then use the accumulated ethanol as an energy source. But how and when did *Saccharomyces* invent the diauxic shift? And which genes does it involve?



Two evolutionary hypotheses with different fates

Remember Susumu Ohno and his Random Breakage Model? Ohno also hypothesized that there exist rare evolutionary events called **whole genome duplications**, or **WGDs**, which duplicate an entire genome. He proposed this **WGD Model** in 1970, when there was absolutely no evidence to support it. Yet he believed that a WGD would be needed at the time of a critical evolutionary innovation for a species to implement some revolutionary new function, such as the diauxic shift.

For example, imagine the time, millions of years ago, when the first fruit-bearing plants had evolved, but no organisms could metabolize the glucose produced by these fruits and take advantage of the produced ethanol. The first species to do so would have possessed an enormous evolutionary advantage, but metabolizing glucose — let alone ethanol — is not a simple task. Rather than creating a new gene here or there, the diauxic shift would have required creating new metabolic pathways with many genes working together. Ohno argued that a WGD would provide a platform for such a revolutionary innovation, since every duplicated gene would have two copies. One copy would be free to evolve without compromising the gene's existing function, which would be carried out by the remaining copy.



The Random Breakage Model and the WGD Model had very different fates. From the time of its proposal, the Random Breakage Model was embraced by biologists, and it became dogma until its refutation in 2003. In contrast, the WGD Model was initially met with skepticism (because only 13% of *S. cerevisiae* genes are duplicated) and did not gain traction for 25 years.

In 1997, Wolfe and Shields provided the first computational arguments in favor of a WGD in *S. cerevisiae*. They argued that the fact that only 13% of *S. cerevisiae* genes are duplicated is not surprising because even if hundreds of genes contribute to an evolutionary innovation after a WGD, most genes are not needed for this innovation. Thus, unneeded duplicate genes will be bombarded by mutations until they turn into pseudogenes and eventually disappear from the genome after millions of years.





Which yeast genes drive the diauxic shift?

One of the many steps yeast performs during fermentation is the conversion of **acetaldehyde** into ethanol. If oxygen becomes subsequently available, then the accumulated ethanol is converted back into acetaldehyde. Both the acetaldehyde-to-ethanol and ethanol-to-acetaldehyde conversions are catalyzed by an enzyme called **alcohol dehydrogenase (Adh)**. In *S. cerevisiae*, Adh activity is encoded by two genes, *Adh₁* and *Adh₂*, which arose from the duplication of a single ancestral gene. The enzyme encoded by *Adh₁* has an elevated ability of producing ethanol, whereas the enzyme encoded by *Adh₂* has an elevated ability of consuming ethanol.

In 2005, Michael Thomson used a multiple alignment of *Adh* genes from various yeast species to reconstruct an ancient *Saccharomyces* gene. This gene showed a preference to convert acetaldehyde to ethanol, which resembled the behavior of *Adh₁*. Thomson therefore concluded that before the WGD in *Saccharomyces*, alcohol dehydrogenase was mainly involved in the generation, not consumption, of ethanol. After the WGD, *Adh₁* carried out its original function, whereas *Adh₂* was free to help power the diauxic shift.

STOP and Think: How would you find the rest of the genes in *S. cerevisiae* that work together to accomplish the diauxic shift?



Imagine that you were able to monitor all n yeast genes at m time checkpoints on either side of the diauxic shift, resulting in an $n \times m$ **gene expression matrix** E , where E_{ij} is a number representing the expression level of gene i at checkpoint j . The i -th row of E is called the **expression vector** of gene i . Just by looking at the yeast genes' expression vectors, you would observe different patterns of gene behavior with respect to the diauxic shift. You would see genes whose expression hardly changes, genes whose expression rapidly increases before the diauxic shift and decreases afterwards, genes whose expression suddenly increases after the diauxic shift, and so on.

Although this chapter focuses on gene expression with respect to the diauxic shift, expression matrices are commonplace in biological analysis. For example, if the expression vector of a newly sequenced gene is similar to the expression vector of a gene with known function, a biologist may suspect that these genes perform related functions. Also, genes with similar expression vectors may imply that the genes are co-regulated, meaning that their expression is controlled by the same transcription factor. This suggests a “guilt by association” strategy for inferring gene functions by starting from a few genes with known functions and potentially propagating the functions of these genes to other genes with similar expression vectors. Finally, gene expression analysis is important in biomedical studies such as analyzing tissues before and after a drug is administered or contrasting cancerous and non-cancerous cells. For example, expression analysis led to **MammaPrint**, a diagnostic test that determines the likelihood of breast cancer recurrence and is based on the expression analysis of 70 human genes associated with tumor activation and suppression.

Yet across all these applications, the question remains: *what methods do biologists apply to analyze gene expression data?*



Gene expression analysis

In 1997, Joseph DeRisi conducted the first massive gene expression experiment by sampling an *S. cerevisiae* culture seven times at hours -6, -4, -2, 0, +2, +4, and +6, where hour 0 indicates the diauxic shift. Since there are approximately 6,400 genes in *S. cerevisiae*, this experiment resulted in a $6,400 \times 7$ gene expression matrix.

DeRisi's Dataset [_\(\[http://bioinformatics.algorithms.com/data/realdatasets/Clustering/diauxic_raw_ratios_RG.txt\]\(http://bioinformatics.algorithms.com/data/realdatasets/Clustering/diauxic_raw_ratios_RG.txt\)\)](http://bioinformatics.algorithms.com/data/realdatasets/Clustering/diauxic_raw_ratios_RG.txt)

STOP and Think: What technology would you use to generate this matrix?





We have already encountered three technologies that could be used to generate this matrix (see [DETOUR: Measuring Gene Expression](https://stepic.org/lesson/Detour-Measuring-Gene-Expression-10937/) (<https://stepic.org/lesson/Detour-Measuring-Gene-Expression-10937/>)), but none of these technologies had matured by 1997! For this reason, DeRisi had to use **microarrays**, which differ from the DNA arrays that we discussed in a previous chapter (see [DETOUR: Microarrays](https://stepic.org/lesson/Detour-Microarrays-10938/) (<https://stepic.org/lesson/Detour-Microarrays-10938/>) to learn more about microarrays). Microarrays are rarely used today, but the algorithmic approaches deRisi used for microarray analysis work equally well for modern gene expression technologies.

STOP and Think: The figure below visualizes the expression vectors of three yeast genes. Which of these genes do you think is involved in the diauxic shift?

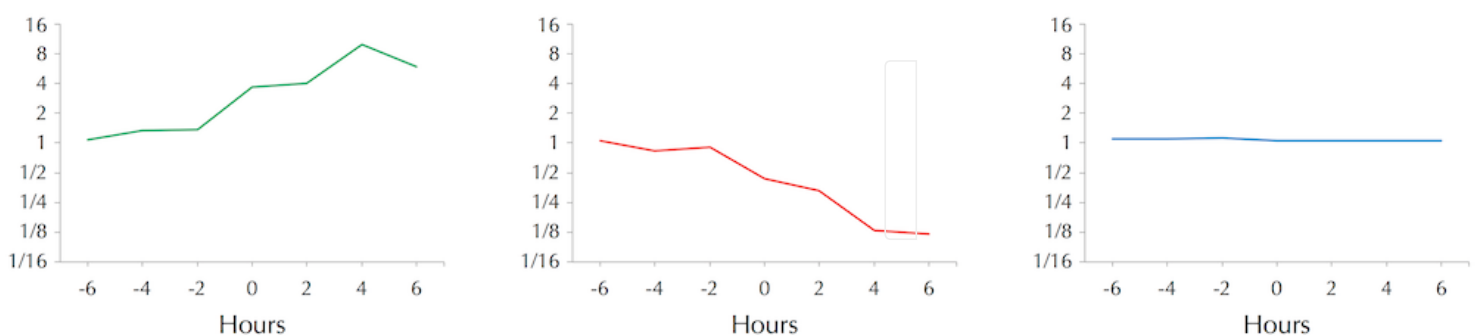


Figure: Expression vectors $(1.07, 1.35, 1.37, 3.70, 4.00, 10.00, 5.88)$, $(1.06, 0.83, 0.90, 0.44, 0.33, 0.13, 0.12)$, and $(1.11, 1.11, 1.12, 1.06, 1.05, 1.06, 1.05)$ of three yeast genes (YLR258W, YPL012W, and YPR055W, respectively) visualized as plots. Each expression vector $(e_1, \dots, e_m)$ is represented as a collection of line segments connecting points (j, e_j) to $(j+1, e_{j+1})$ for each j between 1 and $m-1$. In the DeRisi experiment, the expression level at the initial checkpoint corresponds to the base level of expression; note that it is close to 1 for the three genes. Values above 1 in expression vectors correspond to increased expression, while values below 1 correspond to decreased expression.



Note that the pattern of the expression vector of gene YPR055W remains flat during the diauxic shift. We therefore conclude that this gene is probably not involved in the diauxic shift. On the other hand, the expression of gene YLR258W significantly changes during the diauxic shift, leading us to hypothesize that this gene is involved in the diauxic shift. Indeed, checking the *Saccharomyces* (<http://yeastgenome.org>) Genome Database (<http://yeastgenome.org>) reveals that YLR258W is **glycogen synthase**. This enzyme controls the production of **glycogen**, a glucose polysaccharide that is the main storage vessel for glucose in yeast cells.

In practice, biologists often take the logarithm of expression values (see figure below). After this transformation, positive values of a gene's expression vector correspond to increased expression, and negative values correspond to decreased expression.

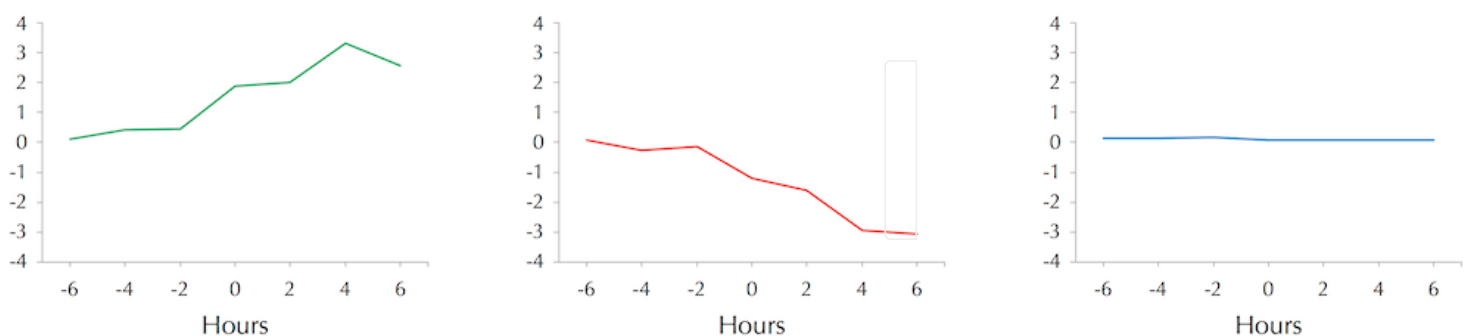


Figure: The expression vectors of the three genes from the previous step with expression levels substituted by their base-2 logarithms: (0.11, 0.43, 0.45, 1.89, 2.00, 3.32, 2.56), (0.09, -0.28, -0.15, -1.18, -1.59, -2.96, -3.08), and (0.15, 0.15, 0.17, 0.09, 0.07, 0.09, 0.07).



The figure below shows an expression matrix of ten yeast genes after taking logarithms.

Gene	Expression Vector						
YLR361C	0.14	0.03	-0.06	0.07	-0.01	-0.06	-0.01
YMR290C	0.12	-0.23	-0.24	-1.16	-1.40	-2.67	-3.00
YNR065C	-0.10	-0.14	-0.03	-0.06	-0.07	-0.14	-0.04
YGR043C	-0.43	-0.73	-0.06	-0.11	-0.16	3.47	2.64
YLR258W	0.11	0.43	0.45	1.89	2.00	3.32	2.56
YPL012W	0.09	-0.28	-0.15	-1.18	-1.59	-2.96	-3.08
YNL141W	-0.16	-0.04	-0.07	-1.26	-1.20	-2.82	-3.13
YJL028W	-0.28	-0.23	-0.19	-0.19	-0.32	-0.18	-0.18
YKL026C	-0.19	-0.15	0.03	0.27	0.54	3.64	2.74
YPR055W	0.15	0.15	0.17	0.09	0.07	0.09	0.07

Figure: A 10×7 submatrix of DeRisi's $6,400 \times 7$ gene expression matrix for ten yeast genes (after taking the base-2 logarithm of each expression value). The genes from the figure on the previous step are colored appropriately.



Clustering yeast genes

Our goal is to **partition** the set of all yeast genes into k disjoint **clusters**. In practice, the number of clusters is not known a priori, and so biologists typically apply clustering algorithms to gene expression data for various values of k , selecting the value of k that makes sense biologically. For simplicity, we will assume that k is fixed. The figure below shows a partition of the genes from the figure on the previous step into three clusters indicating increased, decreased, and flat expression during the diauxic shift.

YLR361C	0.14	0.03	-0.06	0.07	-0.01	-0.06	-0.01
YMR290C	0.12	-0.23	-0.24	-1.16	-1.40	-2.67	-3.00
YNR065C	-0.10	-0.14	-0.03	-0.06	-0.07	-0.14	-0.04
YGR043C	-0.43	-0.73	-0.06	-0.11	-0.16	3.47	2.64
YLR258W	0.11	0.43	0.45	1.89	2.00	3.32	2.56
YPL012W	0.09	-0.28	-0.15	-1.18	-1.59	-2.96	-3.08
YNL141W	-0.16	-0.04	-0.07	-1.26	-1.20	-2.82	-3.13
YJL028W	-0.28	-0.23	-0.19	-0.19	-0.32	-0.18	-0.18
YKL026C	-0.19	-0.15	0.03	0.27	0.54	3.64	2.74
YPR055W	0.15	0.15	0.17	0.09	0.07	0.09	0.07

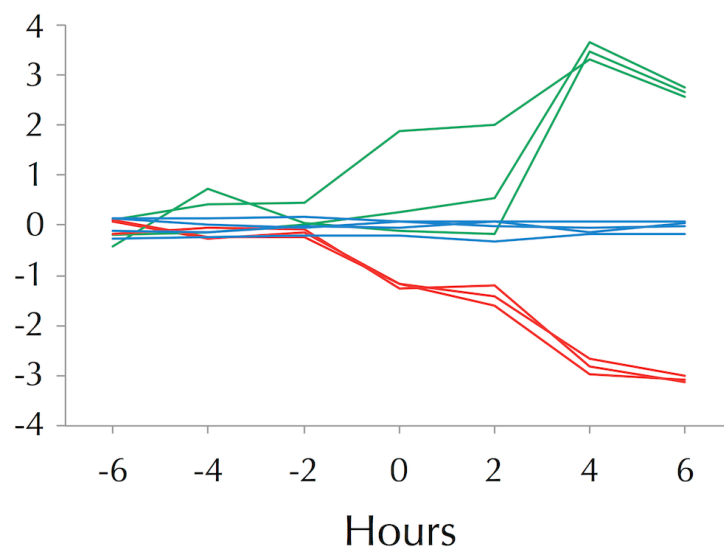


Figure: (Top) The rows of the gene expression matrix from the figure on the previous step partitioned into three clusters. Green genes exhibit increased expression, red genes exhibit decreased expression, and blue genes exhibit flat behavior and are unlikely to be associated with the diauxic shift. (Bottom) The rows of the matrix visualized as plots.



Although the diauxic shift is an important event in the life of *S. cerevisiae*, it has no bearing on most of the yeast's functions. We therefore suspect that most *S. cerevisiae* genes have flat behavior during the diauxic shift, and we would like to exclude these genes from further consideration, thus reducing the size of the expression matrix.

The expression levels of most yeast genes hardly change before and after the diauxic shift (the blue genes in the 10 x 7 yeast gene matrix, reproduced below). These genes also have the property that all of their expression vector values are very close to zero. In our analysis, we will exclude genes with expression vectors whose values are all between -2.3 and 2.3. Doing so reduces the original 6,400 yeast gene dataset to a dataset containing 230 genes whose expression changes significantly around the diauxic shift.

Abridged Diauxic Shift Dataset

(http://bioinformaticsalgorithms.com/data/realdatasets/Clustering/230genes_log_expression.txt)

You can see in the figure on the previous step that gene YLR258W has a different pattern of change than gene YGR043C, indicating that dividing the 230 yeast genes into just two clusters (i.e., those with increasing and decreasing expression levels) may be too simplistic. Our goal is to cluster these genes based on similar patterns of behavior.

YLR361C	0.14	0.03	-0.06	0.07	-0.01	-0.06	-0.01
YMR290C	0.12	-0.23	-0.24	-1.16	-1.40	-2.67	-3.00
YNR065C	-0.10	-0.14	-0.03	-0.06	-0.07	-0.14	-0.04
YGR043C	-0.43	-0.73	-0.06	-0.11	-0.16	3.47	2.64
YLR258W	0.11	0.43	0.45	1.89	2.00	3.32	2.56
YPL012W	0.09	-0.28	-0.15	-1.18	-1.59	-2.96	-3.08
YNL141W	-0.16	-0.04	-0.07	-1.26	-1.20	-2.82	-3.13
YJL028W	-0.28	-0.23	-0.19	-0.19	-0.32	-0.18	-0.18
YKL026C	-0.19	-0.15	0.03	0.27	0.54	3.64	2.74
YPR055W	0.15	0.15	0.17	0.09	0.07	0.09	0.07

Figure: The element with the largest absolute value in each expression vector is shown in bold.



The Good Clustering Principle

To identify groups of genes with similar expression, we will think of an expression vector of length m as a point in m -dimensional space; genes with similar expression vectors will therefore correspond to nearby points. Ideally, clusters should satisfy the following common-sense principle, which is illustrated in the figure below.

Good Clustering Principle: *Every pair of points from the same cluster should be closer to each other than any pair of points from different clusters.*

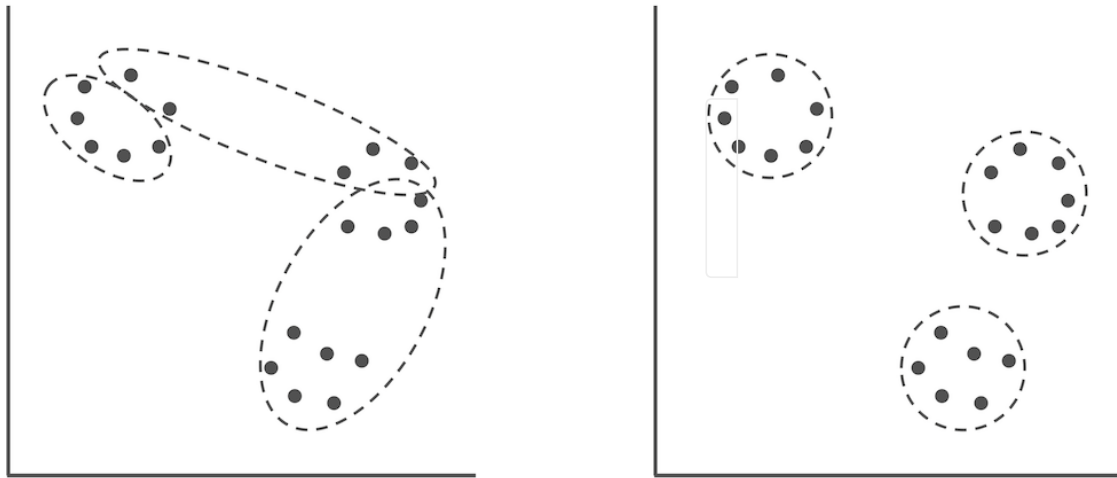


Figure: (Left) A partition of twenty points into three clusters that do not satisfy the Good Clustering Principle. (Right) A different partition of these points that does satisfy the Good Clustering Principle.



We have therefore embedded gene expression analysis within the algorithmic problem of **partitioning** a collection of n points in m -dimensional space into k **clusters**, which will be our focus in the remainder of the chapter.

Good Clustering Problem: *Partition a set of points into clusters.*

Input: A set of n points in m -dimensional space and an integer k .

Output: A partition of the n points into k clusters
satisfying the Good Clustering Principle.

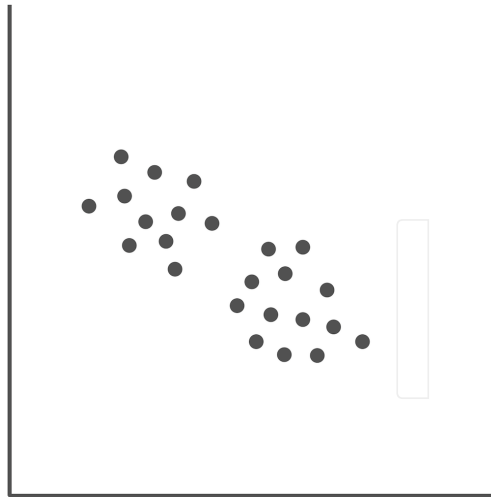
Exercise Break: Form ten points in two-dimensional space by taking the fourth and seventh columns of the 10 x 7 yeast matrix reproduced below. How should these points be partitioned into three clusters?

YLR361C	0.14	0.03	-0.06	0.07	-0.01	-0.06	-0.01
YMR290C	0.12	-0.23	-0.24	-1.16	-1.40	-2.67	-3.00
YNR065C	-0.10	-0.14	-0.03	-0.06	-0.07	-0.14	-0.04
YGR043C	-0.43	-0.73	-0.06	-0.11	-0.16	3.47	2.64
YLR258W	0.11	0.43	0.45	1.89	2.00	3.32	2.56
YPL012W	0.09	-0.28	-0.15	-1.18	-1.59	-2.96	-3.08
YNL141W	-0.16	-0.04	-0.07	-1.26	-1.20	-2.82	-3.13
YJL028W	-0.28	-0.23	-0.19	-0.19	-0.32	-0.18	-0.18
YKL026C	-0.19	-0.15	0.03	0.27	0.54	3.64	2.74
YPR055W	0.15	0.15	0.17	0.09	0.07	0.09	0.07



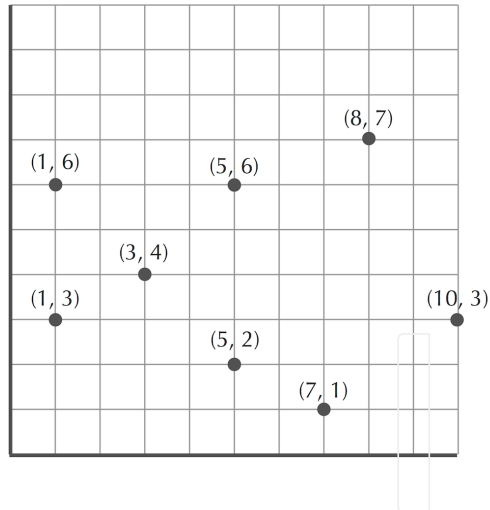
Exercise Break: Compute the number of partitions of n points into two non-empty clusters.

Exercise Break: The eye naturally divides the points in the figure below into two clusters. Unfortunately, these clusters do not satisfy the Good Clustering Principle; in fact, no such partition of these points into two clusters exists. Design a polynomial algorithm to check whether there is a solution of the Clustering Problem.





STOP and Think: The figure below (bottom) shows eight data points in two-dimensional space. How would you partition these points into three clusters? How can we transform the Clustering Problem into a well-defined computational problem?





Clustering as an optimization problem

Rather than thinking about clustering as dividing data points into k clusters, we will instead try to select a set *Centers* of k points that will serve as the **centers** of these clusters. We would like to choose *Centers* so that they minimize some distance function between *Centers* and *Data* over all possible choices of centers. But how should this distance function be defined?

First, we define the **Euclidean distance** between points $v = (v_1, \dots, v_m)$ and $w = (w_1, \dots, w_m)$ in m -dimensional space, denoted $d(v, w)$, as the length of the line segment connecting these points,

$$d(v, w) = \sqrt{\sum_{j=1}^m (v_j - w_j)^2}.$$

Next, given a point *DataPoint* in multi-dimensional space and a set of k points *Centers*, we define the distance from *DataPoint* to *Centers*, denoted $d(\text{DataPoint}, \text{Centers})$, as the Euclidean distance from *DataPoint* to its closest center,

$$d(\text{DataPoint}, \text{Centers}) = \min_{\text{all points } x \text{ from } \text{Centers}} d(\text{DataPoint}, x).$$

The length of the segments in the figure below correspond to $d(\text{DataPoint}, \text{Centers})$ for each point *DataPoint*.

We now define the distance between all data points *Data* and centers *Centers*. This distance, denoted $\text{MaxDistance}(\text{Data}, \text{Centers})$, is the maximum of $d(\text{DataPoint}, \text{Centers})$ among all data points *DataPoint*,

$$\text{MaxDistance}(\text{Data}, \text{Centers}) = \max_{\text{all points } \text{DataPoint} \text{ from } \text{Data}} d(\text{DataPoint}, \text{Centers}).$$

In the figure below, this distance corresponds to the length of the red segment.

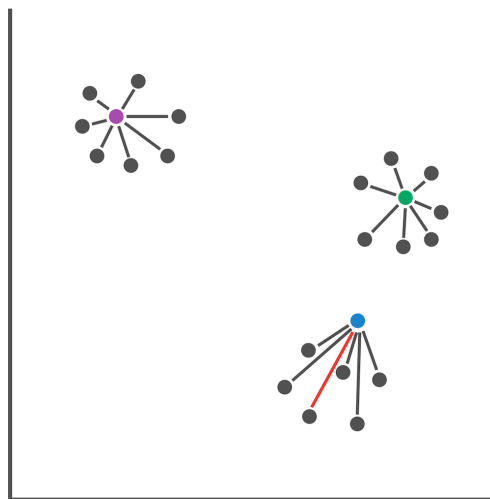


Figure: The collection of points *Data* (shown as black points) along with three centers forming a set *Centers* (shown as colored points). For each point *DataPoint* in *Data*, $d(\text{DataPoint}, \text{Centers})$ is equal to the length of the segment connecting it to its nearest center. $\text{MaxDistance}(\text{Data}, \text{Centers})$ is equal to the length of the longest such segment, which is shown in red.



We can now formulate a well-defined clustering problem.

***k*-Center Clustering Problem: Given a set of data points, find *k* centers minimizing the maximum distance between these data points and centers.**

Input: A set of points *Data* and an integer *k*.

Output: A set *Centers* of *k* centers that minimize the distance $MaxDistance(DataPoints, Centers)$ over all possible choices of *k* centers.

Exercise Break: How would you select a center in the case of only a single cluster (i.e., when $k = 1$)?





Farthest First Traversal

Although the k -Center Clustering Problem is easy to state, it is NP -Hard. The **Farthest First Traversal** heuristic, whose pseudocode is shown below, selects centers from the points in *Data*. It begins by selecting an arbitrary point in *Data* as the first center and iteratively adds a new center as the point in *Data* that is farthest from the centers chosen so far, with ties broken arbitrarily (see figure below).

FarthestFirstTraversal(*Data*, k)

DataPoint $\leftarrow$ an arbitrary point from *Data*

Centers $\leftarrow$ the set consisting of the single point *DataPoint*

while $|Centers| < k$

DataPoint $\leftarrow$ the point in *Data* maximizing $d(DataPoint, Centers)$

 add *DataPoint* to *Centers*

return *Centers*

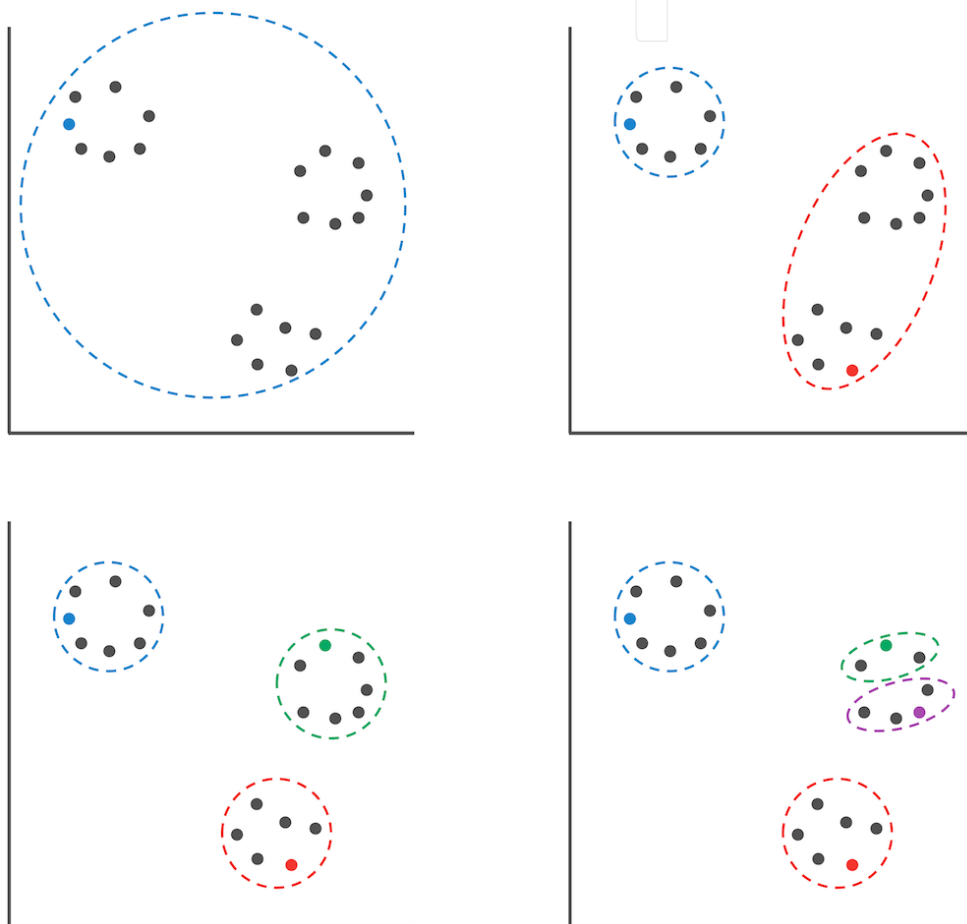


Figure: Applying **FarthestFirstTraversal**. (Top left) An arbitrary point from the dataset (shown in blue) is selected as the first center. All points belong to a single cluster. (Top right) The red point is selected as the second center, since it is the farthest from the blue point. (Bottom left) After computing each data point's minimum distance to each of the first two centers, we find that the point with the largest such distance is the green point, which becomes the third center. (Bottom right) The fourth center is shown in purple.



CODE CHALLENGE: Implement the **FarthestFirstTraversal** clustering heuristic.

Input: Integers k and m followed by a set of points $Data$ in m -dimensional space.

Output: A set $Centers$ consisting of k points (centers) resulting from applying **FarthestFirstTraversal**($Data, k$), where the first point from $Data$ is chosen as the first center to initialize the algorithm.

Extra Dataset [_ \(http://bioinformaticsalgorithms.com/data/extradatasets/clustering/FarthestFirstTraversal.txt\)](http://bioinformaticsalgorithms.com/data/extradatasets/clustering/FarthestFirstTraversal.txt)

Sample Input:

```
3 2
0.0 0.0
5.0 5.0
0.0 5.0
1.0 1.0
2.0 2.0
3.0 3.0
1.0 2.0
```

Sample Output:

```
0.0 0.0
5.0 5.0
0.0 5.0
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-10926/step/14



Exercise Break: Let $Centers$ be the set of centers returned by **FarthestFirstTraversal**, and let $Centers_{opt}$ be a set of centers corresponding to an optimal solution of the k -Center Clustering Problem. Prove that

$$MaxDistance(Data, Centers) \leq 2 \cdot MaxDistance(Data, Centers_{opt}).$$

Can you find a collection of data points such that the centers returned by **FarthestFirstTraversal** are suboptimal?



FarthestFirstTraversal is fast, and according to the preceding exercise, its solution approximates the optimal solution of the k -Center Clustering Problem; however, this algorithm is rarely used for gene expression analysis. In k -Center Clustering, we selected *Centers* so that these points would minimize $MaxDistance(Data, Centers)$, the maximum distance between any point in *Data* and its nearest center. But biologists are usually interested in analyzing *typical* rather than *maximum* deviations, since maximum deviations may correspond to outliers representing experimental errors (see the figure below).

STOP and Think: Can you devise an alternative scoring function that is more biologically appropriate than $MaxDistance$?

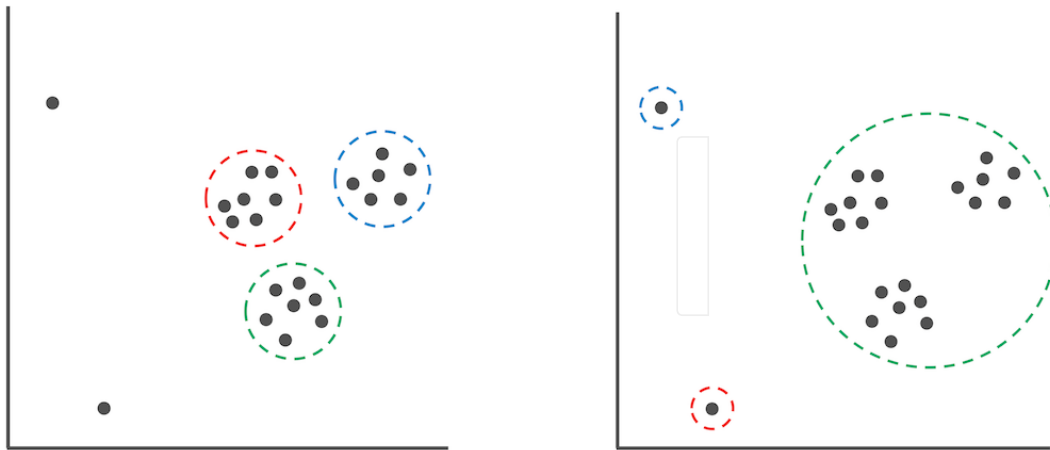


Figure: (Left) A set of data points with three clearly seen clusters and two outliers. (Right) Because **FarthestFirstTraversal** relies on $MaxDistance$ to compute new centers, if we attempt to cluster the data points into three clusters, then regardless of which point is selected as the first center, the two outliers on the left will be selected as centers of single-element clusters, with all remaining data points assigned to a single cluster.

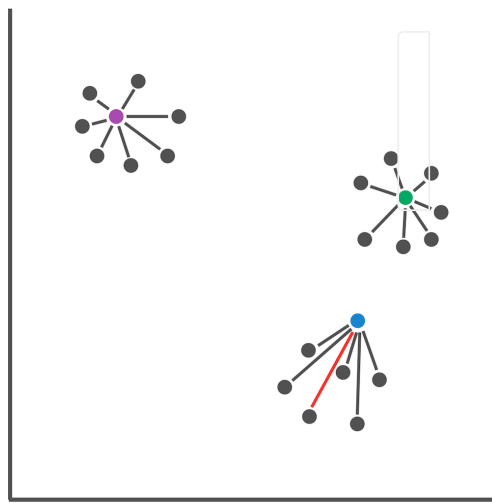


Squared error distortion

To address limitations of *MaxDistance*, we will introduce a new scoring function. Given a set *Data* of n data points and a set *Centers* of k centers, the **squared error distortion** of *Data* and *Centers*, denoted $Distortion(Data, Centers)$, is defined as the mean squared distance from each data point to its nearest center,

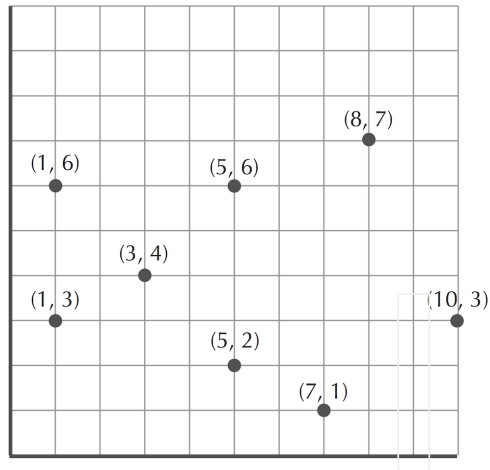
$$Distortion(Data, Centers) = (1/n) \sum_{\text{all points } DataPoint \text{ in } Data} d(DataPoint, Centers)^2 .$$

Note that whereas $MaxDistance(Data, Centers)$ only accounts for the length of the single red segment in the figure below, the squared error distortion accounts for the length of all segments in this figure.





Exercise Break: Compute the values of $MaxDistance(Data, Centers)$ and $Distortion(Data, Centers)$ for the eight data points in the figure below and the three centers $(3, 4.5)$, $(6, 1.5)$, and $(9, 5)$. How do these values differ if the centers are instead $(5/3, 13/3)$, $(6.5, 6.5)$, and $(22/3, 2)$?





Squared Error Distortion Problem: *Compute the squared error distortion of a set of data points with respect to a set of centers.*

Input: A set of points *Data* and a set of centers *Centers*.

Output: The squared error distortion $Distortion(Data, Centers)$.

CODE CHALLENGE: Solve the Squared Error Distortion Problem.

Input: Integers k and m , followed by a set of centers *Centers* and a set of points *Data*.

Output: The squared error distortion $Distortion(Data, Centers)$.

Extra Dataset [_ \(http://bioinformaticsalgorithms.com/data/extradatasets/clustering/SquaredErrorDistortion.txt\)](http://bioinformaticsalgorithms.com/data/extradatasets/clustering/SquaredErrorDistortion.txt)

Sample Input:

```
2 2
2.31 4.55
5.96 9.08
-----
3.42 6.03
6.23 8.25
4.76 1.64
4.47 4.33
3.95 7.61
8.93 2.97
9.74 4.03
1.73 1.28
9.72 5.01
7.27 3.77
```

Sample Output:

```
18.246
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-10927/step/3



The squared error distortion leads us to the following modification of the k -Centers Clustering Problem.

k -Means Clustering Problem: *Given a set of data points, find k center points minimizing the squared error distortion.*

Input: A set of points $Data$ and an integer k .

Output: A set $Centers$ of k centers that minimize $Distortion(Data, Centers)$ over all possible choices of k centers.





Although the k -Centers and k -Means Clustering Problems may look similar, they may produce different results, as shown in the figure below.

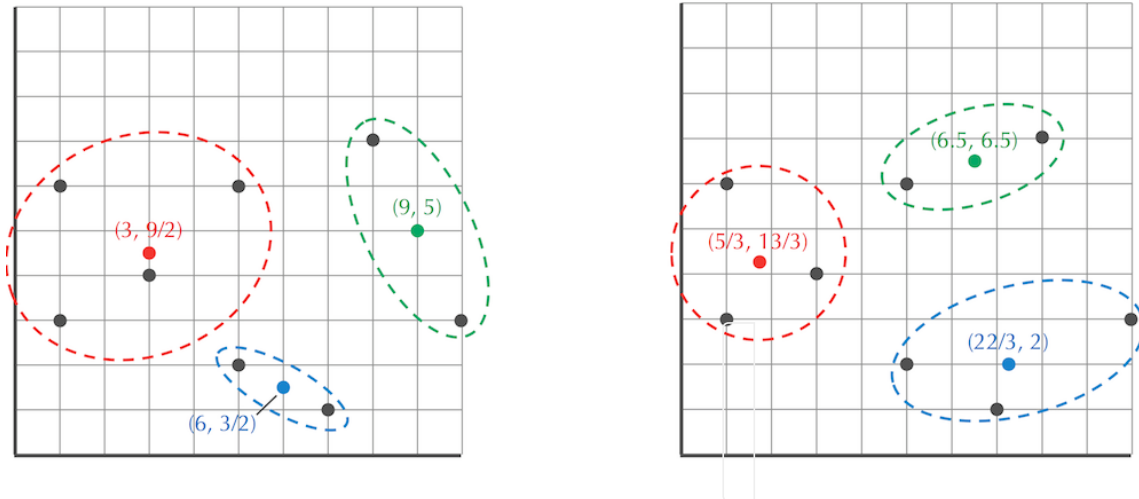


Figure: (Left) The three colored centers solving the k -Centers Clustering Problem for the eight black data points from the figure on the previous step, along with the clusters that they form. (Right) The three colored centers solving the k -Means Clustering Problem for these points, along with the clusters that they form.



The key difference between the k -Centers and k -Means Clustering Problems is that in the latter, the placement of a center is far less affected by outliers (see figure below).

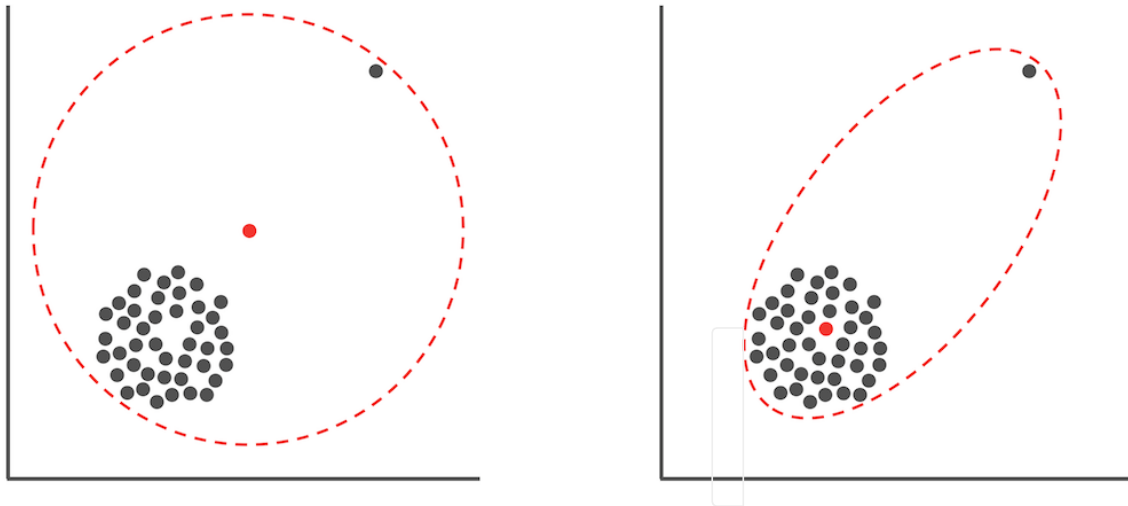


Figure: Center placement varies in different clustering problem formulations. (Left) In the k -Center Clustering Problem, a cluster's center is chosen so that the maximum distance between the center and any point in the cluster is minimized. As a result, the position of the center can be greatly influenced by outliers. (Right) In the k -Means Clustering Problem, the outlier's influence over the placement of the center is much smaller. This is preferable when analyzing biological datasets, in which outliers often correspond to erroneous data.

***k*-means clustering and the center of gravity**

It turns out that the *k*-Means Clustering Problem is *NP*-Hard when $k > 1$. However, when $k = 1$, the *k*-Means Clustering Problem amounts to finding a single center point x that minimizes the squared error distortion. Although we acknowledge that partitioning a set of data points into a single cluster is trivial, it remains unclear how to find a single center minimizing the squared error distortion. We would like to solve this simpler problem because it will help us design a heuristic for the case when $k > 1$.

We define the **center of gravity** of *Data* as the point whose i -th coordinate is the average of the i -th coordinates of all points from *Data*. For example, the center of gravity of the points (3, 8), (8, 3), and (7, 4) is

$$\left(\frac{3 + 8 + 7}{3}, \frac{8 + 3 + 4}{3} \right) = (6, 5)$$



It turns out that the center of gravity is the point x that we are looking for.

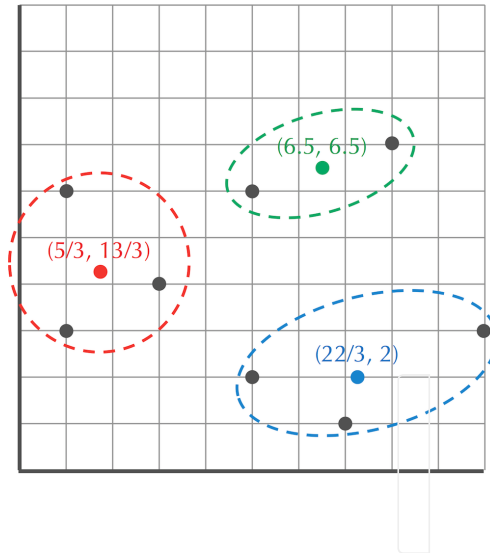
Center of Gravity Theorem: The center of gravity of a set of points Data is the unique point solving the k -Means Clustering Problem for $k = 1$.

For a proof of this theorem, see [DETOUR: Proof of the Center of Gravity Theorem \(https://stepic.org/lesson/Detour-Proof-of-the-Center-of-Gravity-Theorem-10939\)](https://stepic.org/lesson/Detour-Proof-of-the-Center-of-Gravity-Theorem-10939).

Exercise Break: We have already stated that the k -Means Clustering Problem is NP -hard for $k > 1$. However, in the specific case of clustering in one-dimensional space (i.e., when all data points fall on a line), the k -Means Clustering Problem can be solved in polynomial time for any value of k . Design an algorithm for solving the k -Means Clustering Problem in this case.



Exercise Break: Prove that the centers below solve the k -Means Clustering Problem for the black data points shown when $k = 3$.





From centers to clusters and back again

The Lloyd algorithm is one of the most popular clustering heuristics for the k -Means Clustering Problem. It first chooses k arbitrary points *Centers* from *Data* as centers and then iteratively performs the following two steps (see figure on the next step):

- **Centers to Clusters:** After centers have been selected, assign each data point to the cluster corresponding to its nearest center; ties are broken arbitrarily.
- **Clusters to Centers:** After data points have been assigned to clusters, assign each cluster's center of gravity to be the cluster's new center.

In the figure on the next step, the centers appear to be moving less and less between iterations. We say that the Lloyd algorithm has **converged** if the centers (and therefore their clusters) stop changing between iterations.

STOP and Think: Can you find a set of data points for which the Lloyd algorithm does not converge?

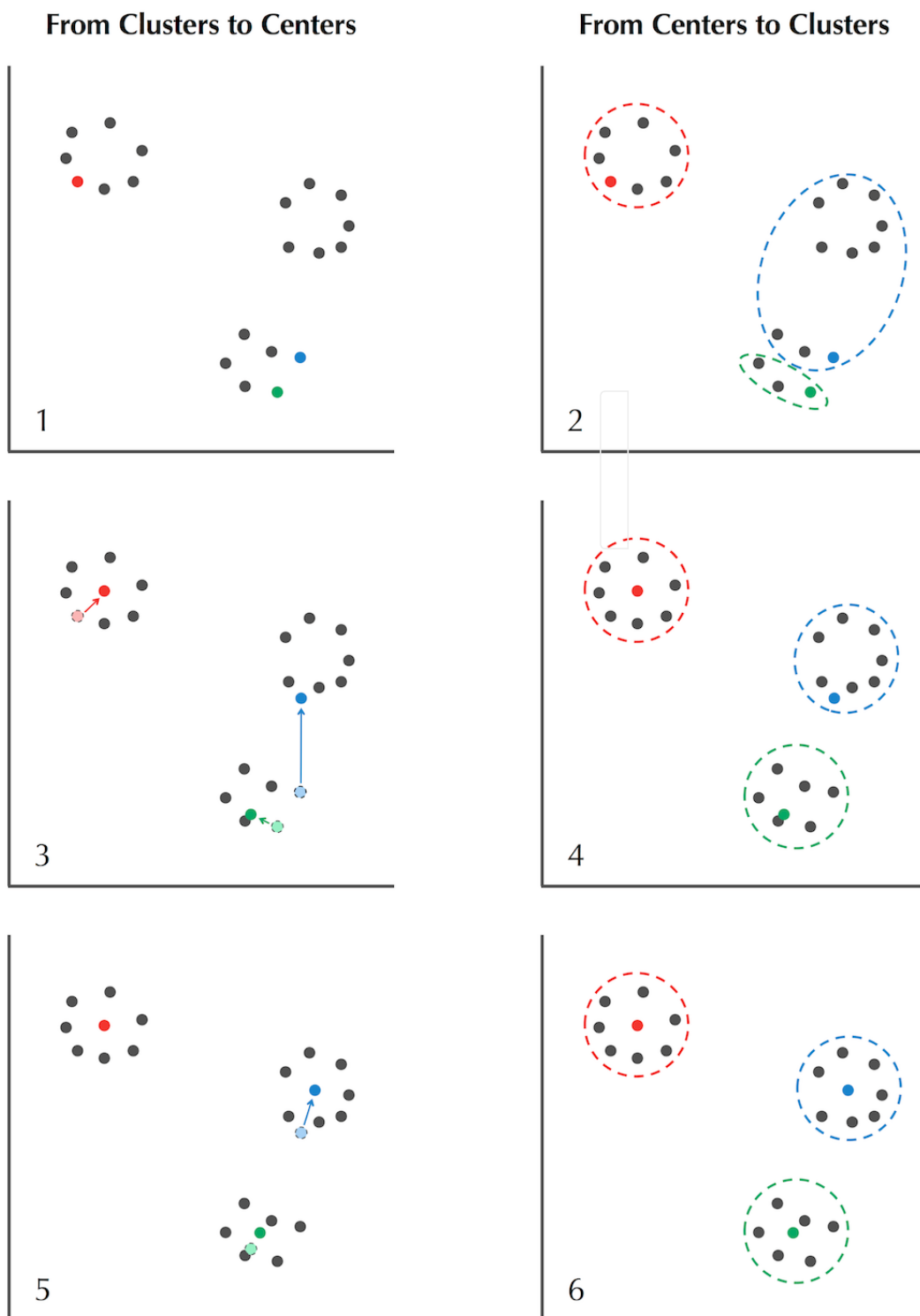


Figure: The Lloyd algorithm in action for $k = 3$. In the top left panel, we select three arbitrary data points as centers, shown as differently colored points. In subsequent panels, we iterate the “Centers to Clusters” step followed by the “Clusters to Centers” step. In the bottom right panel, the Lloyd algorithm has converged.



CODE CHALLENGE: Implement the Lloyd algorithm for k -means clustering.

Input: Integers k and m followed by a set of points *Data* in m -dimensional space.

Output: A set *Centers* consisting of k points (centers) resulting from applying the Lloyd algorithm to *Data* and *Centers*, where the first k points from *Data* are selected as the first k centers.

Extra Dataset <http://bioinformaticsalgorithms.com/data/extradata/clustering/Lloyd.txt>

Sample Input:

```
2 2
1.3 1.1
1.3 0.2
0.6 2.8
3.0 3.2
1.2 0.7
1.4 1.6
1.2 1.0
1.2 1.1
0.6 1.5
1.8 2.6
1.2 1.3
1.2 1.0
0.0 1.9
```

Sample Output:

```
1.800 2.867
1.060 1.140
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-10928/step/3



If the Lloyd algorithm has not converged, the squared error distortion must decrease in any step, according to the following reasoning:

- In a “Centers to Clusters” step, if a data point is assigned to a new center, then this point must be closer to the new center than its previous center. Thus, the squared error distortion must decrease.
- In a “Clusters to Center” step, if a center is updated as a cluster’s center of gravity, then by the Center of Gravity Theorem, the new center is the only point minimizing the squared error distortion for the points in its cluster. Thus, the squared error distortion must decrease.

STOP and Think: Does this reasoning imply that the Lloyd algorithm must converge?





It is not true that the Lloyd algorithm must converge just because the squared error distortion decreases at each step. For example, it could be the case that subsequent decreases in squared error distortion become smaller and smaller, leading to an infinite process (e.g., if the error distortion decreases by $1/2$, then $1/4$, then $1/8$, and so on). The following exercise ensures that such a scenario cannot occur.

Exercise Break: Prove that the number of iterations of the Lloyd algorithm does not exceed the number of partitions of the data set into k clusters.





Exercise Break: Pick your favorite parameter k and run the Lloyd algorithm 1,000 times on the 230-gene diauxic shift dataset, each time initialized with a new set of k randomly chosen centers. Construct a histogram of the squared error distortions of the resulting 1,000 outcomes. How many times did you have to run the Lloyd algorithm before finding the run that scored highest among your 1,000 runs?

Abridged Diauxic Shift Dataset

(http://bioinformaticsalgorithms.com/data/realdatasets/Clustering/230genes_log_expression.txt)





Initializing the Lloyd algorithm

The figure below illustrates that things can go horribly wrong if we do not pay attention to the Lloyd algorithm's initialization step. In the top figure, we select no centers from clump 1, two centers from clump 3, and one center from each of clumps 2, 4, and 5. As shown in the bottom figure, after the first iteration of the Lloyd algorithm, all points in clumps 1 and 2 will be assigned to the red center, which will move approximately halfway between clumps 1 and 2. The two centers in clump 3 will divide the points in that clump into two clusters. And the centers in clumps 4 and 5 will move toward the middle of these clumps. The Lloyd algorithm will then quickly converge, resulting in an incorrect clustering.

Exercise Break: Compute the probability that at least one of the five clumps in Figure 1.13 will have no centers if five centers are chosen randomly from the data (like in the Lloyd algorithm).

STOP and Think: How would you change the Lloyd algorithm's initialization step to improve the clusters it finds?

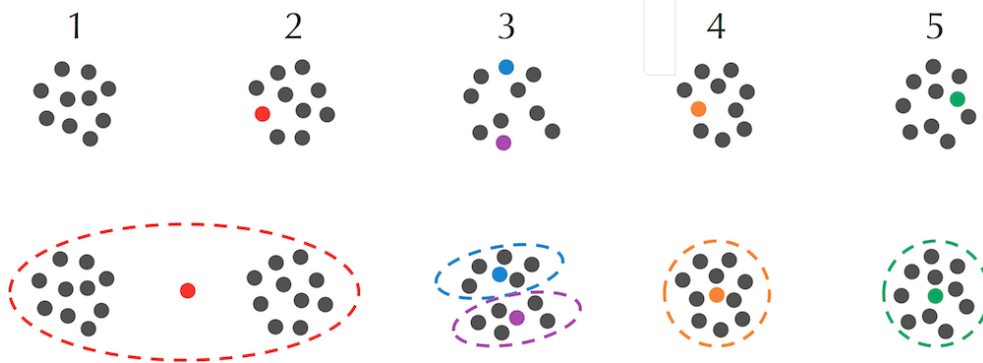


Figure: (Top) Five clumps of ten points in two-dimensional space. The Lloyd algorithm is initialized so that clump 1 contains no centers, clump 3 contains two centers (blue and purple), and each of the other three clumps contains one center (red, orange, and green). (Bottom) Points are clustered according to the center to which they are assigned. The Lloyd algorithm has combined the points in clumps 1 and 2 into a single cluster and split clump 3 into two clusters.

k-means++ Initializer

We have thus far not paid much attention to how initial centers are chosen in the Lloyd algorithm, which selects them randomly. Similarly to *FarthestFirstTraversal*, *k-Means++Initializer* picks k centers one at a time, but instead of choosing the point farthest from those picked so far, it chooses each point at random in such a way that distant points are more likely to be chosen than nearby points. Specifically, the probability of selecting a center *DataPoint* from *Data* is proportional to the squared distance of *DataPoint* from the centers already chosen, i.e., to $d(\text{DataPoint}, \text{Centers})^2$.

For a simple example, say that we have just three data points, and that the squared distances from these points to the existing centers *Centers* are equal to 1, 4, and 5. Then the probability of **k-Means++Initializer** selecting each of these points as the next center is 1/10, 4/10, and 5/10, respectively.

k-Means++Initializer(*Data*, k)

Centers the set consisting of a single randomly chosen point from *Data*

while $|\text{Centers}| < k$

 randomly select *DataPoint* from *Data* with probability proportional to $d(\text{DataPoint}, \text{Centers})^2$

 add *DataPoint* to *Centers*

return *Centers*

STOP and Think: Although **k-Means++Initializer** may also fall into the trap in the figure on the previous step, it does so rarely. Why?

Exercise Break: Power up your implementation of the Lloyd algorithm using **k-Means++Initializer**, and apply it to the 230-gene diauxic shift dataset for varying values of k .

Abridged Diauxic Shift Dataset

(http://bioinformatics.algorithms.com/data/realdatasets/Clustering/230genes_log_expression.txt)



Since selecting the most biologically relevant value of k can be challenging, we will (somewhat arbitrarily) choose to cluster the 230 yeast genes into six clusters (see figure below).

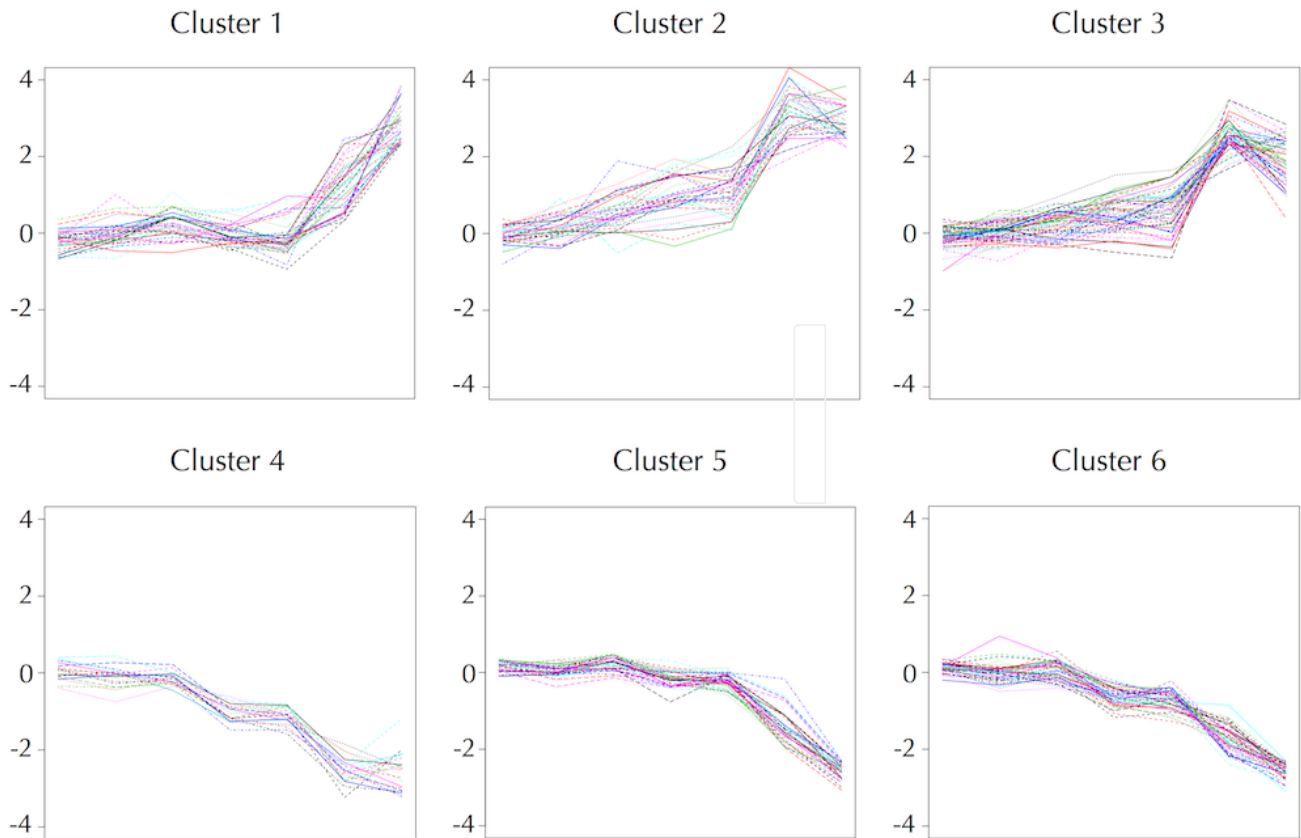


Figure: Applying the Lloyd algorithm (with $k = 6$) to the abridged yeast dataset containing 230 genes results in six clusters containing 37, 36, 58, 19, 36, and 44 genes. Expression vectors for all genes in each of these six clusters are visualized as separate plots.



To reveal the patterns of the expression vectors in each cluster, we average these vectors (see figure below).

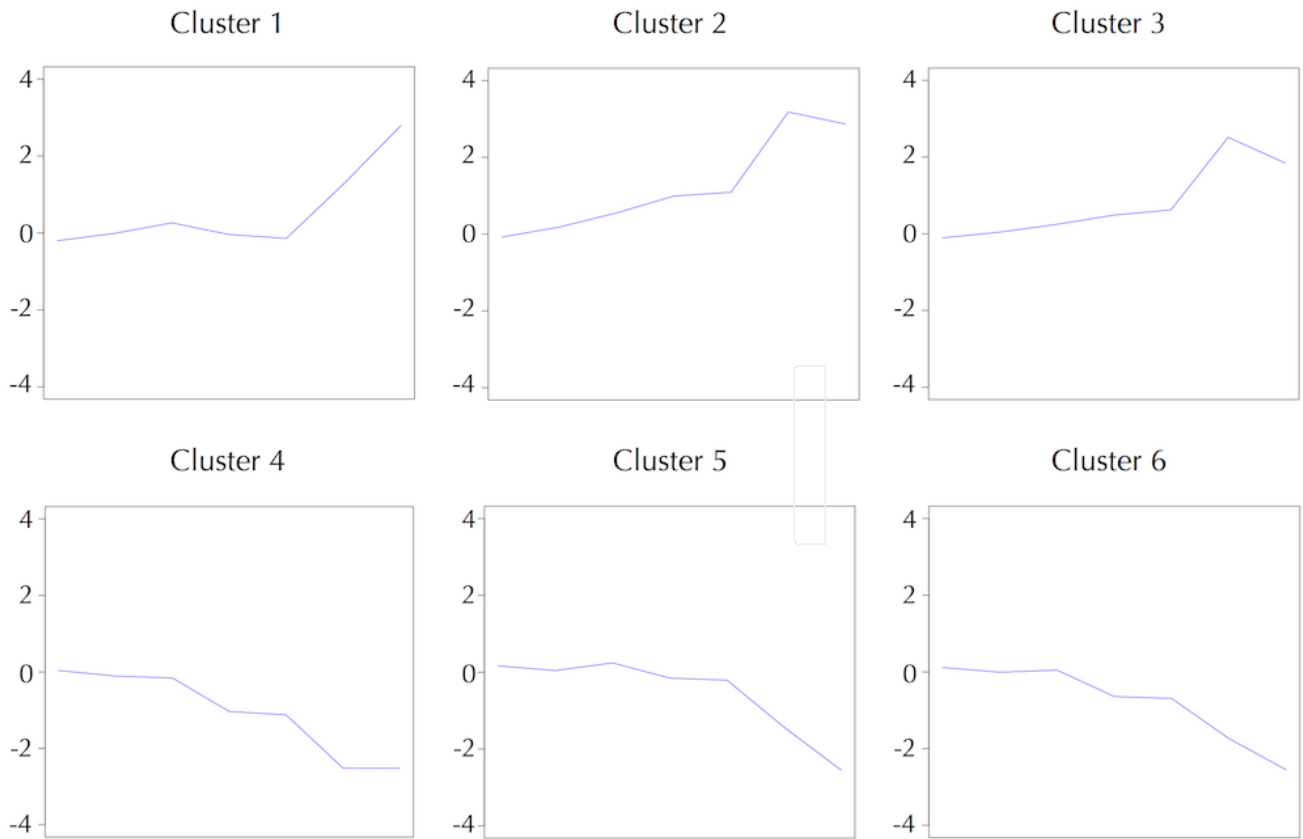


Figure: Averaging the expression vectors in each plot reveals six different types of regulatory behavior.



The yeast plots reveal six patterns of behavior of genes involved in the diauxic shift and raise questions for further biological studies beyond the focus of this chapter. For example, what regulatory mechanisms force the genes in the first cluster to increase their expression? What mechanisms cause the genes in the fourth cluster to decrease their expression? And how do these changes contribute to the diauxic shift?

Exercise Break: The clustering of the entire 6,400-gene yeast dataset into six clusters implies a clustering of the 230 genes. How does this implied clustering compare to the clustering shown on the previous steps?

Unabridged Diauxic Shift Dataset

http://bioinformaticsalgorithms.com/data/realdatasets/Clustering/diauxic_raw_ratios_RG.txt





In the main text, we mentioned that we have encountered three technologies that could be used to measure gene expression. First, in a mass spectrometry experiment (see chapter Antibiotics), biologists generate a set of spectra and match them against a proteome. The number of spectra matching peptides from a given protein offers a proxy for the expression level of this protein. To estimate the protein expression from this proxy, bioinformaticians must account for the varying length of proteins, poor fragmentation of some peptides (leading to difficulties in their identification), and other practical concerns.

Second, in an **RNA sequencing** experiment, we generate reads from a **transcriptome**, or all RNA transcripts present in a cell. By estimating the quantity of each protein-coding RNA transcript in a sample, we obtain a proxy for the expression of the resulting protein. A number of processes affect protein production in the cell in addition to transcription, such as translation, post-translational modifications, and protein degradation. These additional factors can muddle the correlation between the quantity of a transcript and the expression of its corresponding protein.

Third, we can use DNA arrays (see chapter Motifs) carrying probes (k-mers) aimed at each gene in a species of interest. Each probe is characterized by an intensity, which offers a proxy for the number of transcripts of a given gene present in a sample. A deficiency of DNA arrays is that they only target the identification of known transcripts and often fail to evaluate unknown transcripts. For example, many cancers are caused by rare mutations and would go undetected when using DNA arrays. As a result, RNA sequencing is often more attractive in cancer studies, and it is now the dominant technology for analyzing gene expression.



The microarrays that DeRisi used to study the diauxic shift were manufactured as follows. After capturing many RNA transcripts expressed in yeast cells, DeRisi converted each RNA transcript to **complementary DNA (cDNA)** using an enzyme called **reverse transcriptase**, and spotted these cDNAs on a glass slide. He then hybridized the cDNA against fluorescently labeled RNA from a sample of interest in order to measure the expression levels of various yeast genes.

The amount of cDNA printed on each spot of the microarray can vary greatly, a complication that DeRisi needed to address in order to ensure that fluorescence intensities could be compared across spots and across arrays. He therefore hybridized two samples corresponding to two different timestamps to each array (see figure below). He then labeled the samples with different colors of fluorescent dyes so that the samples could be distinguished by image-processing software.

The expression values obtained from a microarray are represented as the ratio of fluorescent intensities of the two samples. Thus, expression is measured as relative changes in the expression of individual genes between samples and time points. For example, if a gene's expression value is 2, then this gene's expression is twice as large in the first sample; if the expression value is $1/3$, then the expression is three times as large in the second sample. Following DeRisi, researchers commonly take the logarithm of these expression ratios to generate expression matrices.

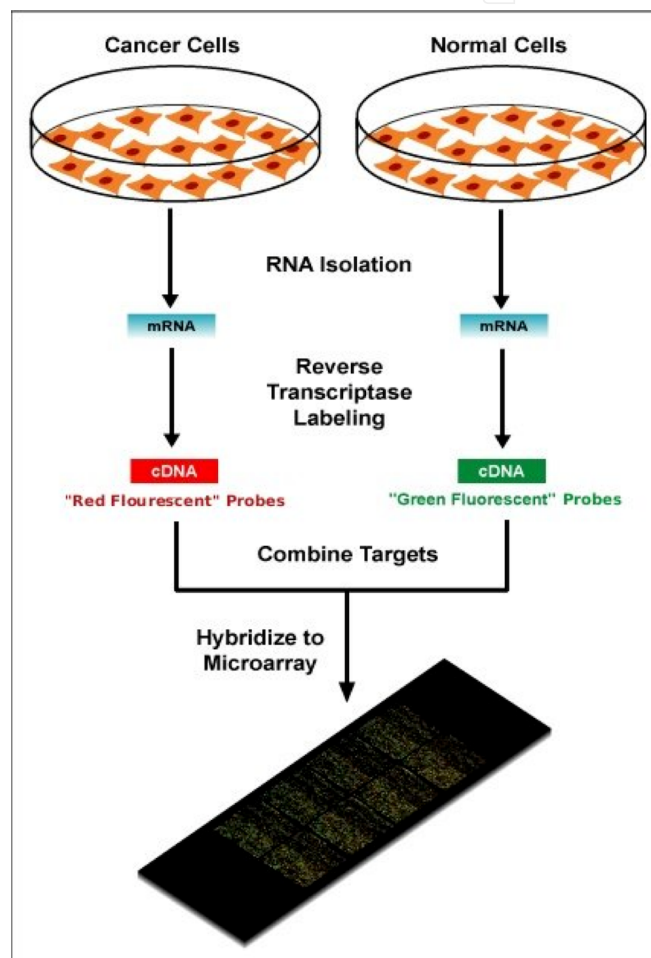


Figure: A microarray against which two samples of fluorescently labeled RNA (red and green) has been hybridized.



Note that when $k = 1$, the k -Means Clustering Problem is equivalent to finding a single center point x that minimizes the sum of squared distances from x to points in $Data$.

Our goal is to show that the center of gravity of a set of points $Data$ is the unique point that $Distortion(Data, x)$ over all possible centers x . Since the squared Euclidean distance between $DataPoint = (DataPoint_1, \dots, DataPoint_m)$ and center $x = (x_1, \dots, x_m)$ is equal to $\sum_{1 \leq j \leq m} (DataPoint_j - x_j)^2$, we have that

$$\begin{aligned} Distortion(Data, x) &= \frac{1}{n} \sum_{\text{all points } DataPoint \text{ in } Data} d(DataPoint, x)^2 \\ &= \frac{1}{n} \sum_{\text{all points } DataPoint \text{ in } Data} \sum_{j=1}^m (DataPoint_j - x_j)^2 \\ &= \frac{1}{n} \sum_{j=1}^m \sum_{\text{all points } DataPoint \text{ in } Data} (DataPoint_j - x_j)^2 \end{aligned}$$

The last line of this formula implies that we can independently minimize $Distortion(Data, x)$ in each of m dimensions by minimizing each of the m expressions

$$\sum_{\text{all points } DataPoint \text{ in } Data} (DataPoint_j - x_j)^2$$

Each of these expressions is a concave-up quadratic function of a single variable x_j . Thus, we can find the minimum of this function by finding where its derivative is equal to zero:

$$\sum_{\text{all points } DataPoint \text{ in } Data} -2 \cdot (DataPoint_j - x_j) = 0.$$

The only solution of this equation is given by

$$x_j = 1/n \sum_{\text{all points } DataPoint \text{ in } Data} DataPoint_j$$

implying that the center's j -th coordinate is the mean value of the j -th coordinates of the data points. In other words, the unique solution of the k -Means Clustering Problem for $k = 1$ is simply the center of gravity of all data points.